COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

07

PACKAGE 8 · GUIDE

Start here

A guided first session, tool ladder and study plan

Undergraduate computer-engineering core

Modules 31-35 · Gate G

Edition: September 28, 2026

A guided self-study curriculum. Supplied examples are learning references, not completed learner projects. Simulation is the default; physical hardware is optional.

Contents

Page references and PDF bookmarks are linked for navigation.

Package 8 / Volume 7 3
Open these in order 3
Before the first lab 3
First session: approximately two hours 3
Commands: run from the extracted package root 4
Tool ladder and honest evidence 4
Planning the volume 5
Completion and scoring 5
Safety and accessibility 6
What to submit at the end 6

Package 8 / Volume 7

Computer Engineering and Digital Design continues the Computer Science and Computer Engineering for Games program. It preserves Modules 31-35, lesson IDs 31.1-35.4, the 60 original core questions and Gate G. It adds longer teaching explanations, 20 explicit lab procedures, 40 transfer questions, reproducible examples and review materials.

This is an undergraduate computer-engineering core inside a larger undergraduate-to-master's self-study program. It is not an accredited degree. The examples teach reasoning and verification; downloading them does not complete the learner's assignments.

Open these in order

1. Open `OPEN_COURSE.html` for the searchable offline reading edition and local progress tracker. No server is required. Export progress regularly because browser storage on local files is not portable or guaranteed.
2. Read `docs/VOLUME_07_HANDBOOK.pdf` for the expanded lessons. Each lesson ends with its original three-question quiz and two new transfer questions.
3. Work in `docs/LAB_AND_PROJECT_WORKBOOK.pdf`. Keep your own code in a new `work/` directory; preserve the supplied examples for comparison.
4. Attempt `docs/ASSESSMENTS.pdf` before using the separate `docs/ANSWER_KEY.pdf`. Read `docs/GATE_G_CAPSTONE.pdf` early enough to reuse module work.

Every PDF has an editable Markdown source in `source/`. `source/course_data.json` preserves the machine-readable lesson map. `PACKAGE_MANIFEST.json` and `SHA256SUMS.txt` inventory the release. Reference directories are examples, not student submissions.

Before the first lab

Required prior learning is explicit, not implied by the package number. Module 31 assumes Modules 06-10 and algebra/calculus competence. Module 32 also assumes Modules 15-16. Module 33 assumes Python and C++ foundations. Module 34 assumes Module 16 and Modules 32-33. Module 35 assumes Module 21 and Modules 31-34. Revisit missing prerequisites before continuing.

You should be able to rearrange equations with units, manipulate signed binary numbers, write a small Python test and distinguish a function from stored state. You do not need to own an FPGA board. You do need to install an HDL simulator and synthesis tools later to complete the RTL and implementation-evidence requirements. Python-only work is a useful first session, not a substitute for the full volume.

First session: approximately two hours

Spend the first 15 minutes reading this guide and creating `work/31_1_divider.md` from the lab-record template. Write your prediction for the 5 V, 2 kohm/3 kohm divider before running anything.

Spend 20 minutes on the preflight and Python reference checks below. Read the output rather than merely checking that a command exited. Missing HDL tools do not prevent this first circuit-model exercise.

Spend 40 minutes on Lesson 31.1 and the loaded-divider derivation. Work the load case by hand, run the reference, then change the load. Explain current and power conservation in your own words.

Spend 30 minutes completing the lab table and deliberately introducing the series-versus-parallel mistake in a private copy. Keep both the failing and repaired test output.

Spend the last 15 minutes answering the three core questions and two transfer questions without the key. Record which explanation you could not reconstruct. A small, honest notebook is a better starting artifact than marking every lesson complete.

Commands: run from the extracted package root

Use Python 3.10 or newer. The supplied Python examples require only the standard library. On systems where the command is `python`, substitute it for `python3`. These commands do not install software or change machine settings.

```
python3 tools/preflight.py
python3 tools/run_python_tests.py
python3 -m reference.python.circuits
python3 -m reference.python.input_demo
python3 -m reference.python.video
python3 tools/generate_cpu_vectors.py
```

`preflight.py` reports actual installed versions. `run_python_tests.py` writes its result and complete test log under `evidence/`. The circuit output is analytical; the input CSV and PPM frame are software-reference artifacts. The vector generator writes assembled G16 programs and expected retirement traces; it does not execute an RTL simulator.

Tool ladder and honest evidence

The authoring environment executed 74 Python unit tests successfully. Those tests include 32 directed G16 programs, seeded random arithmetic, exhaustive small logic/arithmetic cases, queue models and peripheral/raster checks. An additional generated comparison set contains those 32 directed programs and 20 seeded random programs.

Icarus Verilog, Verilator, Yosys, ngspice and formal tools were unavailable in the authoring environment. Included HDL, SPICE and synthesis runners therefore have **NOT_RUN** status, not a passing result. The formal harness is a starter, not a proof certificate. Review `evidence/VALIDATION_REPORT.md` for the exact release scope.

Install tools using their official documentation and record the version actually selected. Do not assume a current online manual matches an older local package. The package uses Icarus with `-g2012` for its reference simulation runner, generic Yosys synthesis, ngspice for optional circuit cross-

checks, and a SymbiYosys/Boolector starter for formal exercises. Verilator and cocotb are assigned study and valid alternative testbench tools; they are not required by the included Icarus runner. [R32, R33, R34, R50, V7-R02, V7-R03]

After the corresponding tools are installed:

```
python3 tools/run_hdl.py
python3 tools/run_synthesis.py
python3 tools/run_spice.py
# Formal starter: inspect the harness and assumptions before running.
cd starter/formal
sby -f fifo.sby
```

Missing required executables make the three Python runners return exit code 2 with a recorded NOT_RUN status. A real failure returns a failure code. Do not edit these statuses to obtain a passing checklist. Run output overwrites the associated evidence files; preserve the original release separately when comparing environments.

Planning the volume

The original estimate is **450-720 focused hours**, including readings, labs, module projects and Gate G. The following split is a planning aid, not extra hours or academic credit.

Module	Focus	Planning hours
31	Circuits and electronic foundations	75-115
32	Digital logic, arithmetic and timing	75-115
33	RTL, testbenches, formal reasoning and synthesis	95-155
34	Processor model, RTL and architectural verification	110-180
35	Game-oriented integration and Gate G	95-155
Total	Includes module projects and Gate G	450-720

At 10 focused hours per week, this estimate spans approximately 45-72 weeks; at 20 hours, approximately 23-36 weeks. These are arithmetic planning scenarios, not promises. A lesson often needs several sessions. Advance on evidence rather than elapsed time.

Completion and scoring

For each lesson, finish the assigned study, satisfy the lab's pass criteria and earn at least 5/6 on the original core quiz. Each core question is worth two points: two for a correct, justified answer, one for a substantially correct but incomplete answer, zero for an incorrect or missing answer. Numerical answers need units where applicable.

The two transfer questions add a separate four-point diagnostic. Aim for at least 3/4 and revisit weaknesses; they do not silently replace or renumber the original core questions. Keep the first attempt and the corrected explanation. The browser marks core lesson completion from study, lab and core score; it tracks transfer scores separately.

A module additionally requires its project review. Gate G has a separate 100-point rubric and critical blockers. Passing Gate G does not remove the Module 34 processor requirement. A reviewer should ask for a live change, a failed case and an explanation of evidence boundaries.

Safety and accessibility

Required circuits are simulated. Any physical extension must use reviewed low-voltage educational hardware, a qualified mentor, a current-limited supply where appropriate, a documented common-reference plan and verified I/O ratings. Never open mains-powered supplies, displays or battery packs for this course. Stop before energizing an uncertain connection.

Use short reading sessions and the browser's scalable text or a PDF reader's zoom. The package contains no required videos, remote telemetry, paid board purchase or account registration. Save progress exports and lab notes in your own storage. A peer or mentor is particularly useful for timing, CDC and electrical-safety review.

What to submit at the end

Submit your own circuit notebook; logic, fixed-point and FSM evidence; FIFO RTL and verification; processor reference/RTL comparison including at least 20 directed programs and reproducible random cases; and the integrated Gate G subsystem. Include exact commands, versions, seeds, raw logs, unresolved limitations and a short oral review record. Supplied expected traces are starting references, not proof that your implementation is correct.